

Proma - Object Oriented Design

Oct 1, 2026 · @SM

Proma is modelled as 14 core classes in a layered MERN design: a `User` hierarchy (Admin, Team Lead, Team Member) works on `Project`s made of `Task`s and `Milestone`s, and every task event flows through an Observer-based `NotificationService`. Each class below traces back to FR-1 to FR-9 of the SRS.

1. Identifying the classes

Classes were found by noun-verb analysis of the SRS (FR-1 to FR-9), the use case diagram and Decision Table D.1: nouns became classes or attributes, verbs became methods.

Noun / concept in SRS	Source	Design decision
User, Team Lead, Team Member, System Administrator	2.3 User Classes, use case actors	Abstract class <code>User</code> with subclasses <code>TeamLead</code> , <code>TeamMember</code> , <code>Admin</code>
Team, role	FR-9	Class <code>Team</code> ; enum <code>Role</code>
Project (create, edit, archive)	FR-2	Class <code>Project</code> ; enum <code>ProjectStatus</code>
Task (create, assign, status)	FR-3, FR-4	Class <code>Task</code> ; enums <code>TaskStatus</code> , <code>Priority</code>
Deadline, milestone, reminder	FR-5	<code>deadline</code> attribute on <code>Task</code> ; class <code>Milestone</code> ; class <code>DeadlineScheduler</code>
Dashboard, analytics	FR-6	Class <code>DashboardService</code> (computed, not stored)
Notification	FR-7, Decision Table D.1	Class <code>Notification</code> , <code>NotificationService</code> , interface <code>NotificationRule</code>
Comment	FR-8	Class <code>Comment</code> ; interface <code>Commentable</code>

Noun / concept in SRS	Source	Design decision
JWT login, session token	FR-1, NFR-3	Class <code>AuthService</code> , class <code>AuthToken</code>

Enumerations used across the model

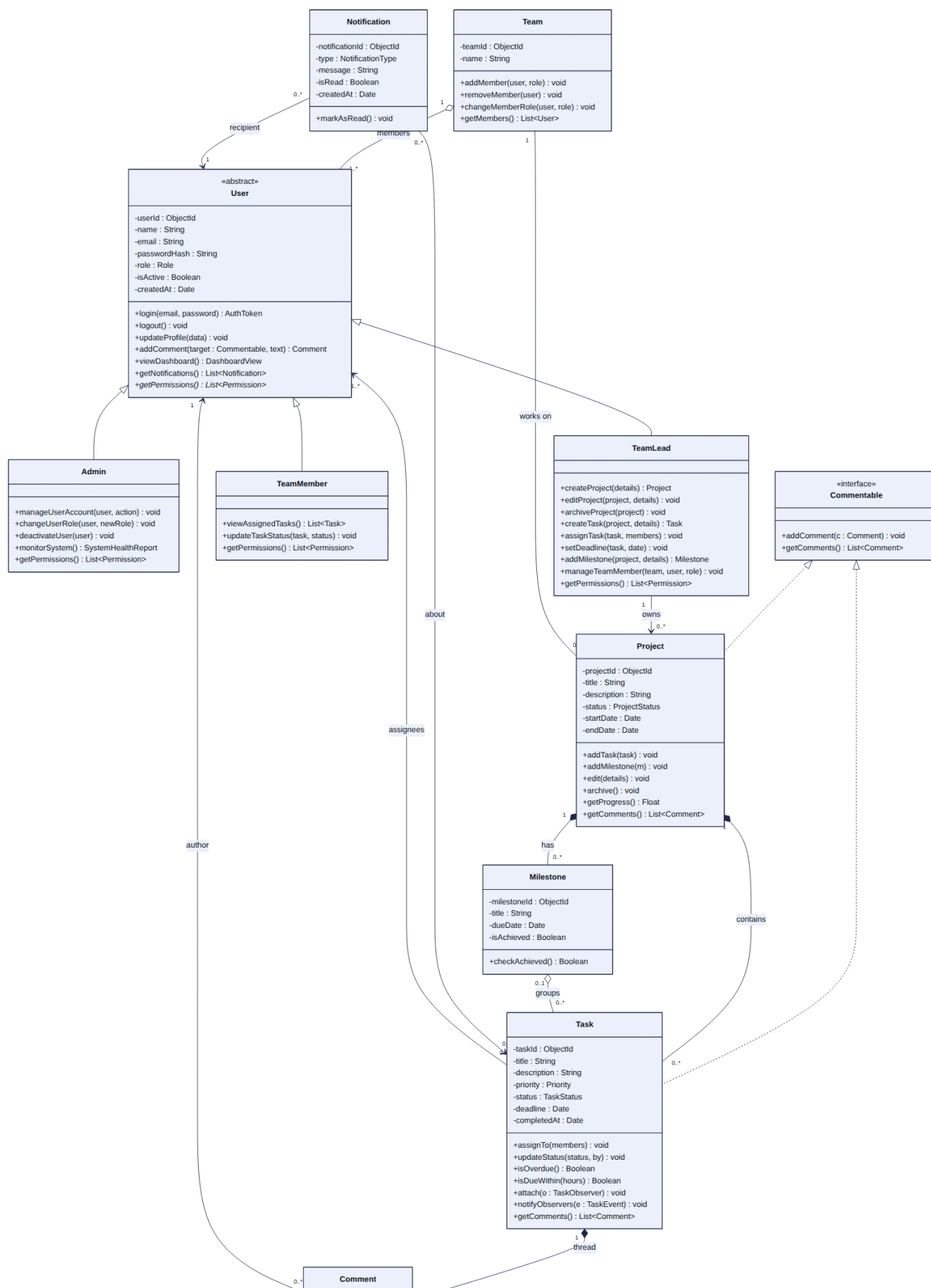
Enumeration	Values
Role	ADMIN, TEAM_LEAD, TEAM_MEMBER
TaskStatus	TO_DO, IN_PROGRESS, IN_REVIEW, COMPLETED
Priority	LOW, MEDIUM, HIGH
ProjectStatus	ACTIVE, ARCHIVED
NotificationType	TASK_ASSIGNED, STATUS_UPDATED, NEW_COMMENT, DEADLINE_APPROACHING, TASK_OVERDUE
EventType	ASSIGNED, STATUS_CHANGED, COMMENT_ADDED, DEADLINE_CHECK

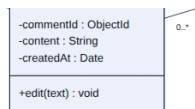
The SRS does not fix the task statuses; the four above are a design choice. Overdue is a computed flag (`isOverdue()`), not a status, so a task can be both In Progress and overdue.

2. Class diagrams

The model is split into two diagrams so each stays readable: the domain model (what Proma stores) and the service model (how it behaves).

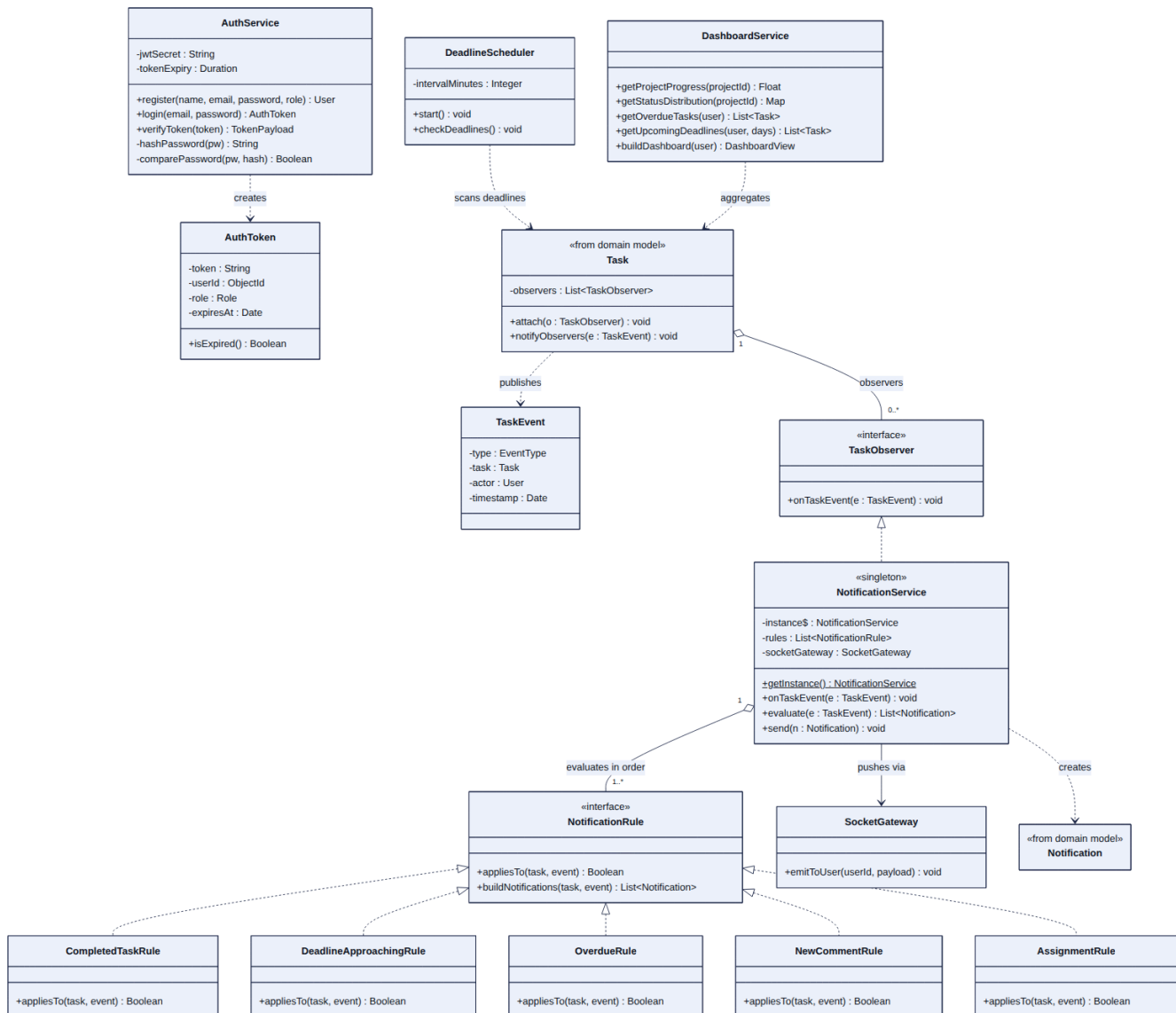
Figure 2.1 - Domain class diagram





User is abstract; its three subclasses each override `getPermissions()`. A Project is composed of its Task s and Milestone s (deleting a project deletes them), while a Team only aggregates User s (removing a member does not delete the user).

Figure 2.2 – Service class diagram (notifications, authentication, dashboard)



Task is the Observer subject: every assignment, status change or comment publishes a TaskEvent to the singleton NotificationService. That service runs the five NotificationRule classes, one per rule of Decision Table D.1, and pushes results through SocketGateway.

UML notation used: hollow triangle = inheritance; dashed hollow triangle = interface realisation; filled diamond = composition; hollow diamond = aggregation; plain arrow = association; dashed arrow = dependency; - private, + public, underlined = static, italic = abstract.

3. Class specifications

Each class has one clear responsibility; attributes are private and reached only through its methods.

3.1 Domain classes

Class	Kind	Responsibility	Key methods
User	Abstract class	Common identity, credentials and actions shared by every role	<code>login()</code> , <code>logout()</code> , <code>updateProfile()</code> , <code>addComment()</code> , <code>viewDashboard()</code> , <code>getNotifications()</code> , abstract <code>getPermissions()</code>
Admin	Subclass of User	Platform maintenance: user accounts, roles, system health	<code>manageUserAccount()</code> , <code>changeUserRole()</code> , <code>deactivateUser()</code> , <code>monitorSystem()</code>
TeamLead	Subclass of User	Plans work: projects, tasks, deadlines, milestones, team roles	<code>createProject()</code> , <code>archiveProject()</code> , <code>createTask()</code> , <code>assignTask()</code> , <code>setDeadline()</code> , <code>addMilestone()</code> , <code>manageTeamMember()</code>
TeamMember	Subclass of User	Executes assigned tasks and reports progress	<code>viewAssignedTasks()</code> , <code>updateTaskStatus()</code>

Class	Kind	Responsibility	Key methods
Team	Class	Groups users and holds their team role	<code>addMember()</code> , <code>removeMember()</code> , <code>changeMemberRole()</code> , <code>getMembers()</code>
Project	Class, implements Commentable	Owns tasks and milestones; reports its own progress	<code>addTask()</code> , <code>addMilestone()</code> , <code>edit()</code> , <code>archive()</code> , <code>getProgress()</code>
Task	Class, implements Commentable; Observer subject	Unit of work with assignees, status, priority, deadline and comment thread	<code>assignTo()</code> , <code>updateStatus()</code> , <code>isOverdue()</code> , <code>isDueWithin(hours)</code> , <code>attach()</code> , <code>notifyObservers()</code>
Milestone	Class	Checkpoint in a project that groups related tasks	<code>checkAchieved()</code> - true when all its tasks are Completed
Comment	Class	One message in a task or project discussion	<code>edit()</code>
Notification	Class	A stored, per-user alert about a task event	<code>markAsRead()</code>

3.2 Service and support classes

Class	Kind	Responsibility	Key methods
AuthService	Service	Registration, password hashing, JWT issue and verification	<code>register()</code> , <code>login()</code> , <code>verifyToken()</code> , <code>private</code> <code>hashPassword()</code> , <code>comparePassword()</code>

Class	Kind	Responsibility	Key methods
AuthToken	Value object	The signed JWT returned at login, carrying userId and role	<code>isExpired()</code>
NotificationService	Singleton; implements TaskObserver	Receives task events, applies rules, saves and pushes notifications	<code>getInstance()</code> , <code>onTaskEvent()</code> , <code>evaluate()</code> , <code>send()</code>
NotificationRule (+5 rules)	Interface + Strategy classes	One class per Decision Table D.1 rule; decides who is notified	<code>appliesTo()</code> , <code>buildNotifications()</code>
DeadlineScheduler	Service (background job)	Periodically scans active tasks for approaching or missed deadlines	<code>start()</code> , <code>checkDeadlines()</code>
DashboardService	Service	Computes progress %, status split, overdue and upcoming tasks for a user	<code>getProjectProgress()</code> , <code>getStatusDistribution()</code> , <code>getOverdueTasks()</code> , <code>getUpcomingDeadlines()</code> , <code>buildDashboard()</code>
SocketGateway	Adapter	Wraps the WebSocket server used for real-time delivery	<code>emitToUser()</code>

3.3 Key method logic

- `Project.getProgress()` = $\text{completed tasks} \div \text{total tasks} \times 100$ (0 when the project has no tasks).
- `Task.isOverdue()` = $\text{status} \neq \text{COMPLETED}$ and $\text{current time} > \text{deadline}$.
- `Task.isDueWithin(24)` = $\text{status} \neq \text{COMPLETED}$ and $\text{deadline} - \text{now} \leq 24$ hours.

- `Task.updateStatus(status, by)` checks that `by` is an assignee or the project's Team Lead, sets `completedAt` when status becomes COMPLETED, then calls `notifyObservers()`.

4. Relationships and multiplicities

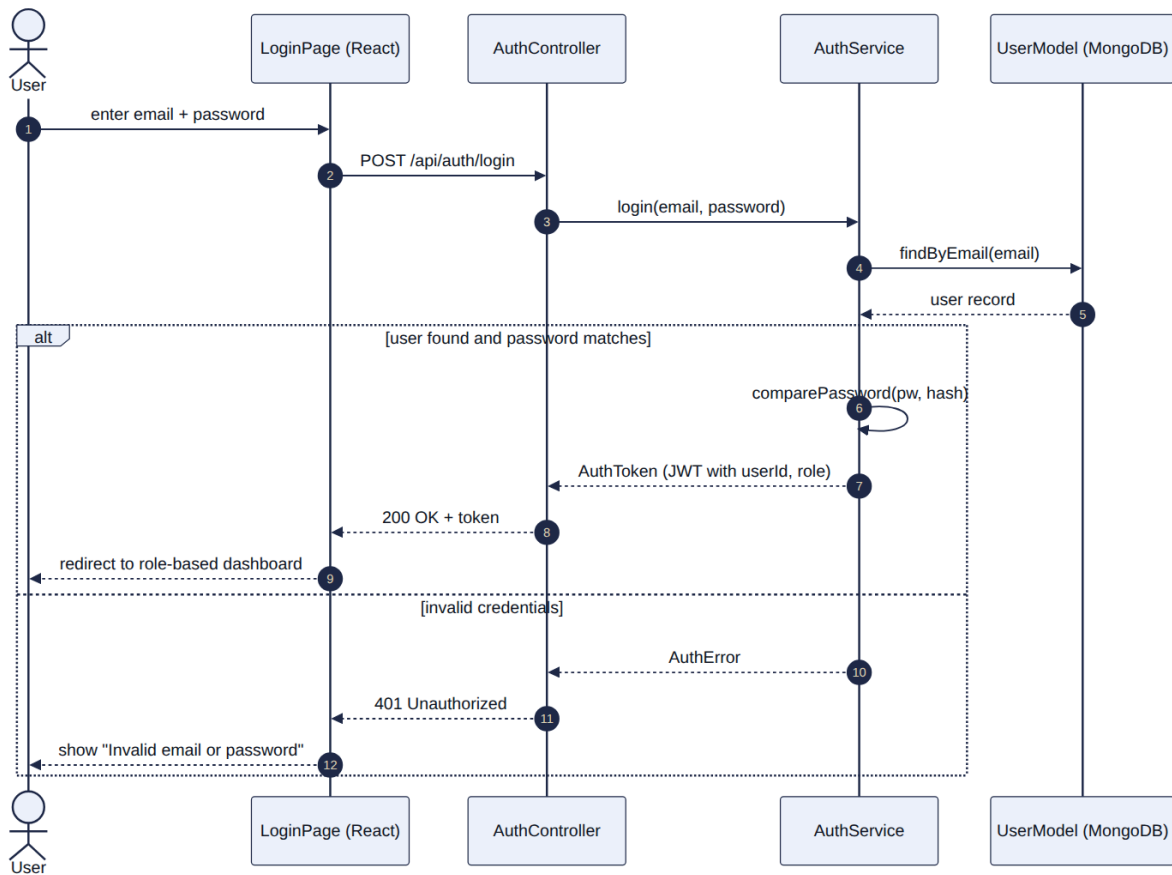
From	Relationship	To	Multiplicity	Meaning
Admin, TeamLead, TeamMember	Inheritance	User	-	Each role is a kind of User and overrides <code>getPermissions()</code>
Task, Project	Realisation	Commentable	-	Both can hold a comment thread
NotificationService	Realisation	TaskObserver	-	Listens to task events
Five rule classes	Realisation	NotificationRule	-	Interchangeable rule strategies
Project	Composition	Task	1 to 0..*	Tasks cannot exist without their project
Project	Composition	Milestone	1 to 0..*	Milestones belong to one project
Task	Composition	Comment	1 to 0..*	Comments are deleted with their task
Team	Aggregation	User	1 to 1..*	Members exist independently of the team
Milestone	Aggregation	Task	0..1 to 0..*	A task may belong to at most one milestone
NotificationService	Aggregation	NotificationRule	1 to 1..*	Rules are evaluated in a fixed order

From	Relationship	To	Multiplicity	Meaning
TeamLead	Association	Project	1 to 0..*	A lead owns many projects
Team	Association	Project	1 to 0..*	A team works on many projects
Task	Association	User (assignees)	0..* to 1..*	FR-3: one or more assignees per task
Comment	Association	User (author)	0..* to 1	Every comment has one author
Notification	Association	User (recipient), Task	0..* to 1, 0..* to 0..1	One recipient; usually about a task
AuthService	Dependency	AuthToken	-	Creates tokens at login
DeadlineScheduler, DashboardService	Dependency	Task	-	Read tasks; do not own them

5. Sequence diagrams

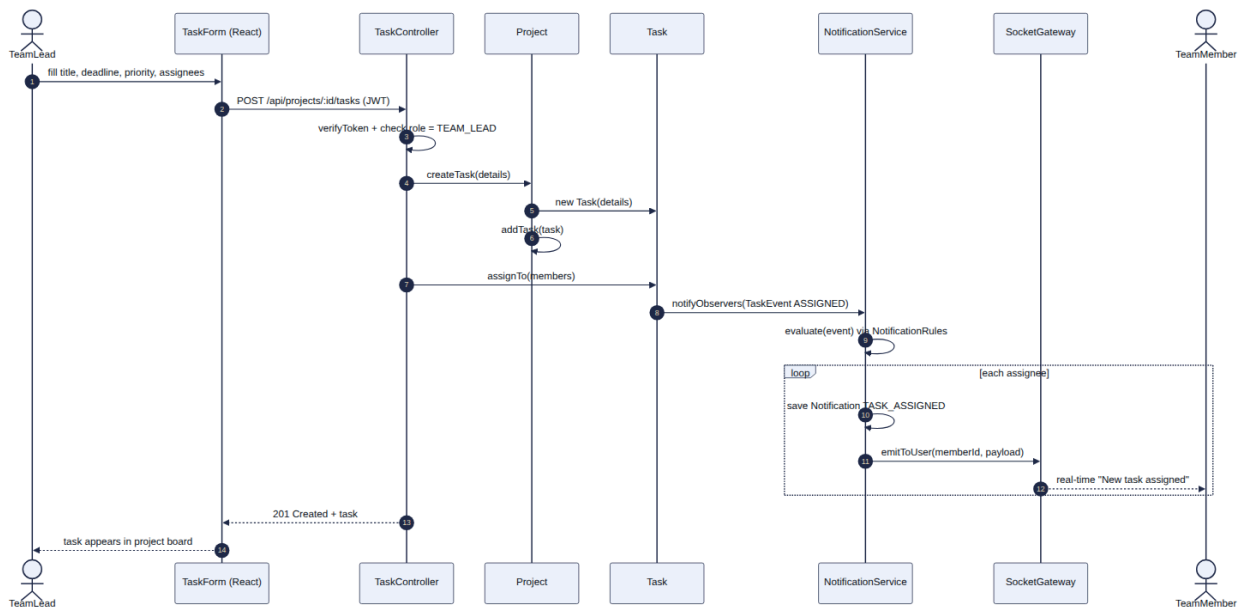
Four sequences cover the high-priority use cases; each shows the objects from Section 2 exchanging messages in time order.

Figure 5.1 - Register & Login (FR-1)



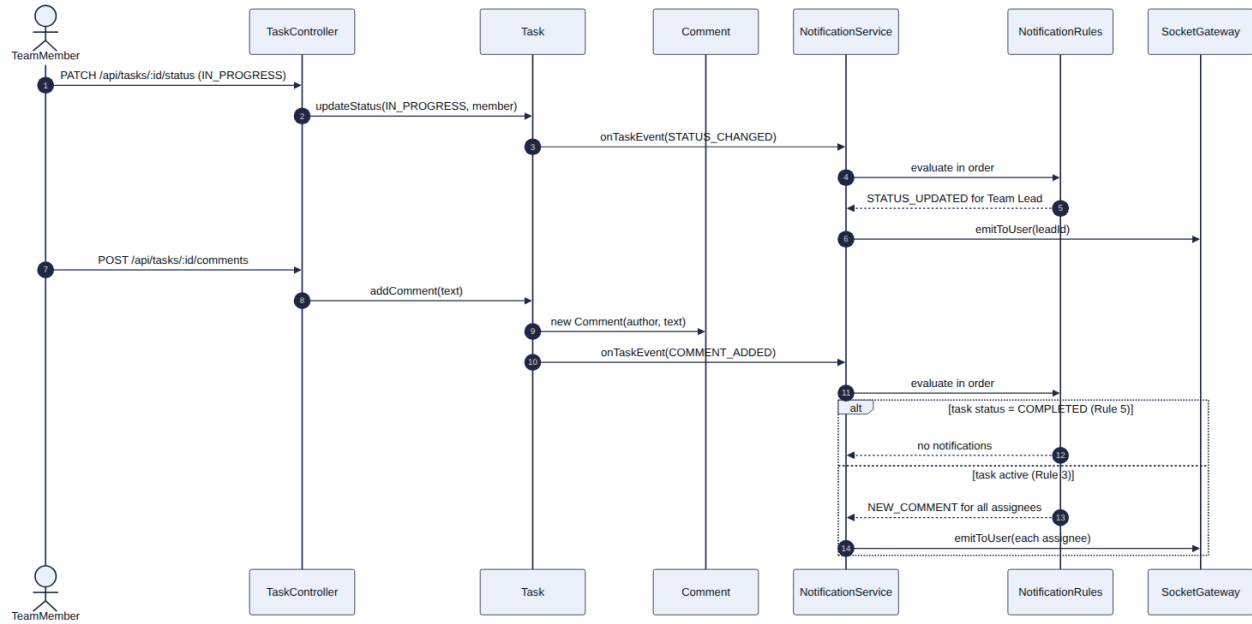
Valid credentials return a signed JWT carrying userId and role; invalid ones return 401 with an error message, matching the SRS acceptance criterion.

Figure 5.2 – Create & Assign Task (FR-3, FR-7)



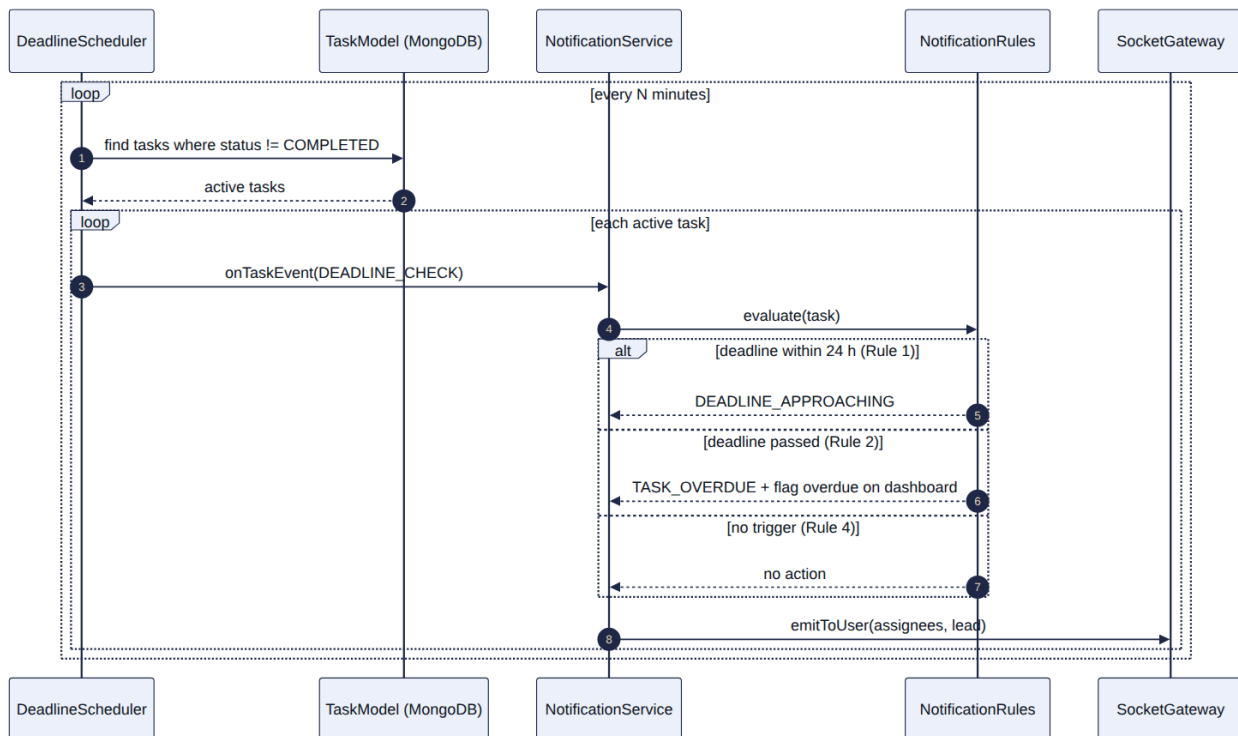
Role is checked before the task is created. Assignment fires one ASSIGNED event, and every assignee receives a real-time notification.

Figure 5.3 – Update Task Status and Add Comment (FR-4, FR-7, FR-8)



A status change notifies the Team Lead. A comment notifies all assignees unless the task is already Completed (Decision Table Rule 5).

Figure 5.4 – Deadline reminder and overdue check (FR-5, FR-6, FR-7)

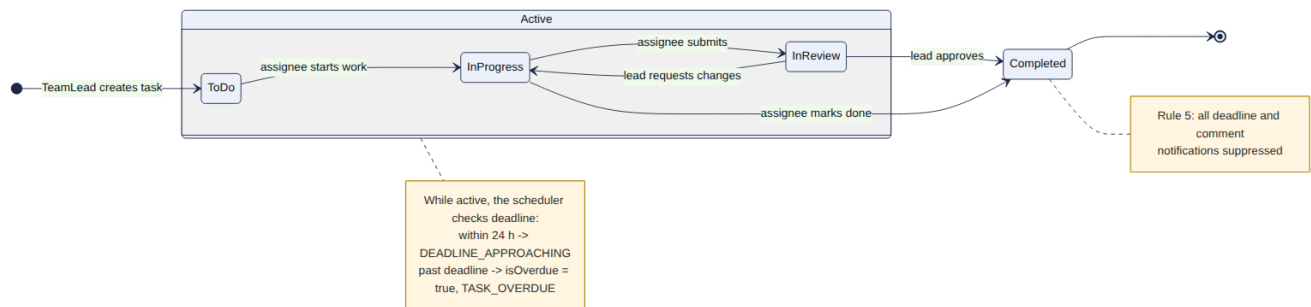


The scheduler skips Completed tasks at the query, so Rule 5 also holds for deadline reminders.

6. Task state diagram

A task moves through four states; Overdue is a condition checked while the task is active, not a separate state.

Figure 6.1 – Task lifecycle

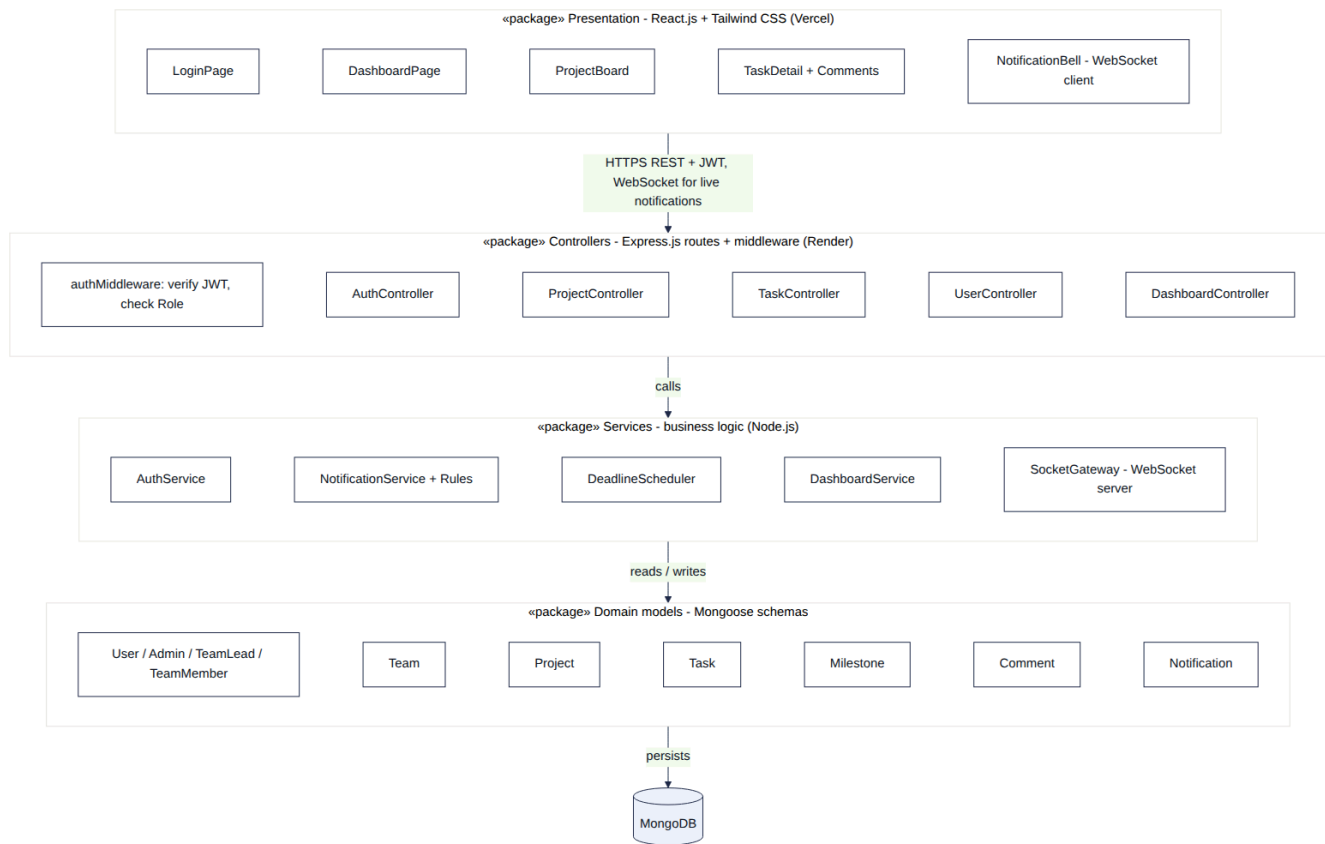


Current state	Event	Next state	Who can trigger
(start)	Task created	To Do	Team Lead
To Do	Work started	In Progress	Assignee
In Progress	Submitted for review	In Review	Assignee
In Review	Changes requested	In Progress	Team Lead
In Review	Approved	Completed	Team Lead
In Progress	Marked done	Completed	Assignee

7. Layered architecture

The classes sit in four packages that follow the MERN stack; each layer calls only the layer below it.

Figure 7.1 – Package diagram



Package	Classes	Technology
Presentation	LoginPage, DashboardPage, ProjectBoard, TaskDetail, NotificationBell	React.js, Tailwind CSS on Vercel
Controllers	authMiddleware, Auth/Project/Task/User/Dashboard controllers	Express.js on Render
Services	AuthService, NotificationService + rules, DeadlineScheduler, DashboardService, SocketGateway	Node.js
Domain models	User hierarchy, Team, Project, Task, Milestone, Comment, Notification	Mongoose schemas on MongoDB

The User hierarchy maps to one MongoDB collection with a `role` discriminator, which keeps queries simple while preserving inheritance in code.

8. OO principles and design patterns

8.1 Core OO principles

Principle	Where it appears in Proma
Abstraction	<code>User</code> , <code>Commentable</code> , <code>TaskObserver</code> and <code>NotificationRule</code> define what is done, not how
Encapsulation	All attributes are private; <code>passwordHash</code> is never exposed, and status changes only happen through <code>Task.updateStatus()</code>
Inheritance	<code>Admin</code> , <code>TeamLead</code> and <code>TeamMember</code> reuse login, profile, comment and dashboard behaviour from <code>User</code>
Polymorphism	<code>getPermissions()</code> returns a different set per role; <code>NotificationService</code> calls <code>appliesTo()</code> on any rule without knowing which one it is

8.2 Design patterns

Pattern	Classes	Why it fits
Observer	<code>Task</code> (subject), <code>TaskObserver</code> , <code>NotificationService</code>	Tasks announce events without knowing who reacts, so new listeners (e.g. email) can be added later
Strategy	<code>NotificationRule</code> + 5 rule classes	Each Decision Table D.1 rule is its own class; a rule can change without touching the others
Singleton	<code>NotificationService</code>	One shared instance holds the rule list and the socket connection
Adapter	<code>SocketGateway</code>	Hides the <code>WebSocket</code> library, so switching to polling (allowed by SRS 5.4) affects one class
MVC / Layered	React views, Express controllers, Mongoose models	Matches the MERN constraint and NFR-6 maintainability

8.3 SOLID check

- **Single responsibility:** authentication, notification, scheduling and analytics each live in their own service.
- **Open/closed:** a new notification rule is a new class, with no edit to `NotificationService`.
- **Liskov substitution:** any `User` subclass can be passed wherever a `User` is expected (e.g. as a comment author).
- **Interface segregation:** `Commentable` and `TaskObserver` each have only two or one method.
- **Dependency inversion:** `Task` depends on the `TaskObserver` interface, not on `NotificationService` directly.

9. Traceability to requirements

Every functional requirement in the SRS is realised by at least one class and method.

Req.	Requirement (short)	Classes	Methods	Diagram
FR-1	Register and log in with JWT, role-based	AuthService, AuthToken, User	<code>register()</code> , <code>login()</code> , <code>verifyToken()</code>	Fig. 5.1
FR-2	Create, edit, archive projects	TeamLead, Project	<code>createProject()</code> , <code>edit()</code> , <code>archive()</code>	Fig. 2.1
FR-3	Create tasks, assign to one or more members	TeamLead, Task	<code>createTask()</code> , <code>assignTo()</code>	Fig. 5.2
FR-4	Assignees update task status	TeamMember, Task	<code>updateTaskStatus()</code> , <code>updateStatus()</code>	Fig. 5.3, 6.1
FR-5	Deadlines, milestones, reminders	Task, Milestone, DeadlineScheduler	<code>setDeadline()</code> , <code>addMilestone()</code> , <code>checkDeadlines()</code>	Fig. 5.4

Req.	Requirement (short)	Classes	Methods	Diagram
FR-6	Progress dashboard with analytics	DashboardService, Project	<code>buildDashboard()</code> , <code>getProgress()</code> , <code>getOverdueTasks()</code>	Fig. 2.2
FR-7	Real-time notifications	NotificationService, NotificationRule, SocketGateway, Notification	<code>onTaskEvent()</code> , <code>evaluate()</code> , <code>emitToUser()</code>	Fig. 5.2-5.4
FR-8	Comments on tasks and projects	Comment, Commentable, User	<code>addComment()</code> , <code>getComments()</code>	Fig. 5.3
FR-9	Add, remove, change role of members	Admin, TeamLead, Team	<code>changeUserRole()</code> , <code>addMember()</code> , <code>removeMember()</code> , <code>changeMemberRole()</code>	Fig. 2.1

References: Proma SRS (IEEE 830), Proma Requirement Elicitation Document, Proma UML Use Case Diagram – Team Proma, KIET Deemed to be University.